Algorithmic Thinking

with Python



Contents

Chapter 1 , The Computer as a Model of Computation 4
1.1 A Brief History of Algorithms.....4
1.2 What Do We Mean by “Computation”? 5
1.3 It All Starts With a Rock 6
1.4 The Diode: A One-Way Street for Electricity.....7
1.5 The Transistor: A Switch With No Moving Parts 7
1.6 Switches Learn to Make Decisions: Logic Gates.....8
1.7 Bits: The Only Two Digits a Computer Needs.....9
1.8 Gates Learn Arithmetic: Building an Adder.....10
1.9 From Adder to ALU 12
1.10 Remembering Things: How a Circuit Holds a Bit 12
1.11 Putting It All Together: Inside a CPU.....13
1.12 From Instructions to Programs.....15
Chapter 1 Questions.....17

Chapter 2 , Problems, Algorithms, and the Problem-Solving Process.....19
2.1 Problems and Computational Problems 19
2.2 What Exactly Is an Algorithm? 19
2.3 Problem-Solving Strategies, in Brief 20
2.4 The Problem-Solving Process 21
2.5 Stage 1: Understanding the Problem.....21
2.6 Stage 2: Formulating a Model 22
2.7 Stage 3a: The Algorithm, in Plain English First 23
2.8 Stage 3b: Drawing It as a Flowchart 24
2.9 Stage 3c: Writing It as Pseudocode 26
2.10 Stage 4: Writing the Program.....27
2.11 Stage 5: Testing the Program 28
2.12 Stage 6: Evaluating the Solution.....29
Chapter 2 Questions.....31

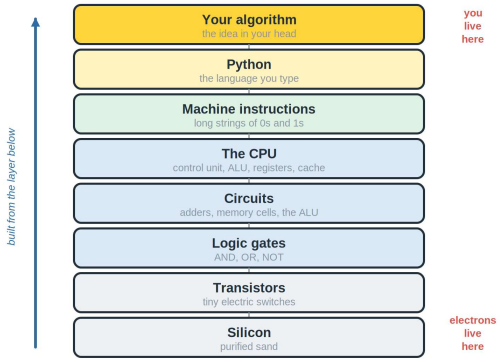
Answer Key 33

CHAPTER 1

The Computer as a Model of Computation

Before we write a single line of Python, it helps to know what we are actually talking to. A computer feels like magic. You type a few words, tap a key, and a machine on your desk (or in your pocket) does exactly what you asked. This chapter takes the magic apart, piece by piece, until there is nothing left but sand, electricity, and a very large number of very simple decisions.

We are going to start from the smallest possible thing , a single electronic component called a diode , and build our way up, layer by layer, until we have something that looks like a real processor. Nobody expects you to design a CPU after reading this. The goal is simpler: once you understand that a computer is really just switches flipping other switches, an awful lot of confusing computer science stops being confusing.



The view from ground level all the way up to your Python code. We are about to climb this ladder from the bottom.



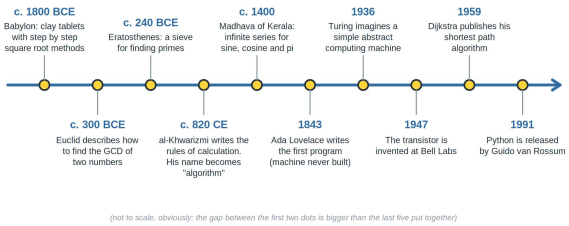
PYTHIA SAYS

Hi. I'm Pythia. I'll be popping in through this book to explain things, complain about semicolons, and occasionally make a joke so bad you'll remember the concept just to forget the joke.

Chapter 1 has no code in it at all. Strange for a Python book, I know. But you can't understand why Python works the way it does until you've met the thing underneath it. Think of this as meeting your computer's grandparents.

1.1 A Brief History of Algorithms

The word “algorithm” is younger than the idea. People have been following step-by-step procedures for thousands of years , a recipe is an algorithm, and so is a knitting pattern. What changed over time is how precisely those procedures were written down, and eventually, what kind of machine could carry them out.



A very rough map of how we got here. The distances are not to scale , most of human history is in that first gap.

A few of these deserve a proper introduction:

- **Al-Khwarizmi** was a 9th century Persian mathematician working in Baghdad. His book on calculation was translated into Latin and his name, garbled through several centuries and languages, became the word "algorithm." He never saw a computer, but he is the reason this book has a title.
- **Madhava of Kerala** led a school of mathematicians in south India around the 14th century who worked out infinite series for trigonometric functions centuries before calculus was "invented" in Europe. If you have ever used sin(x) in Python, a small part of that answer traces back to Kerala.
- **Ada Lovelace** wrote notes in 1843 describing how Charles Babbage's proposed Analytical Engine could compute a sequence of numbers. Nobody built the machine in her lifetime, but her notes are widely regarded as the first computer program, written for a computer that did not yet exist.
- **Alan Turing** in 1936 described an imaginary machine , an infinite strip of tape and a set of simple rules , to answer a question about the limits of mathematics. That imaginary machine turned out to be a complete description of what "computable" means, and it still anchors computer science today.



PYTHIA SAYS

Fun fact: al-Khwarizmi also gave us the word "algebra," from the title of one of his other books. So really, this entire subject is just one very productive man from Baghdad, twelve centuries of interest.

1.2 What Do We Mean by "Computation"?

Strip away the screen, the keyboard, and the fans, and a computer does exactly one kind of thing, over and over, extremely fast: it takes some information, follows a rule, and produces new information. Add two numbers. Compare two numbers. Move a number from one place to another. That is the entire job description.

A model of computation is just a simplified, agreed-upon description of what a machine is allowed to do, so that we can reason about what it can and cannot compute without getting lost in wiring diagrams. There are a few famous ones:

Model	What it says
Turing Machine	An imaginary tape of unlimited length, a head that reads and writes one symbol at a time, and a small table of rules. Slow to think about, but it

Model	What it says
	can, in principle, compute anything that is computable at all. It's the ruler by which “can this be computed” is measured.
Von Neumann architecture	The practical blueprint almost every real computer follows: a processor, memory that holds both the program and the data, and input/output devices, all connected by a shared pathway called a bus.
RAM model	The model programmers actually think in. Every basic operation , add two numbers, compare them, read a memory location , takes roughly the same, constant amount of time. This is what lets us say a loop takes about n steps.

The Church-Turing thesis, a statement almost everyone in the field accepts (though it cannot be formally proved), says that anything a human could compute by following a definite procedure, a Turing machine could compute too. Your laptop, your phone, and a room-sized 1960s mainframe are all, in this sense, equally capable , one is just enormously faster.



PYTHIA SAYS

“Computable” does not mean “easy.” Some problems are computable but would take longer than the age of the universe to actually finish. Others are proven to be uncomputable , no algorithm, however clever, however patient, can solve them. The most famous one is the Halting Problem: no program can look at another arbitrary program and always correctly say whether it will eventually stop or run forever. We will not prove this here , that's a subject for a much drier book , but it is worth knowing the limit exists.

1.3 It All Starts With a Rock

Here is the sentence that annoys electrical engineers the most, and it is completely true: a computer is a rock that we tricked into thinking.

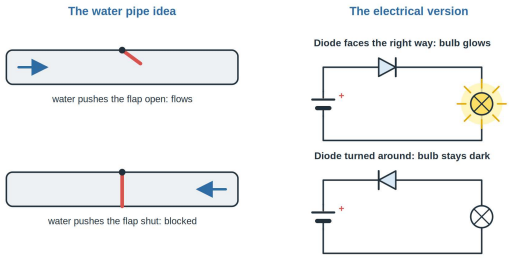
The “rock” is silicon, the second most common element in the Earth's crust, found in sand. On its own, pure silicon is a poor conductor of electricity , not a great insulator like rubber, not a great conductor like copper, just stuck in the middle. That middling behaviour is precisely why it is useful. By adding tiny, carefully chosen impurities to silicon (a process called doping), engineers can turn small regions of it into a material that conducts electricity only under certain conditions and refuses to under others. Get enough of those regions arranged the right way, and you have a switch with no moving parts, controlled entirely by electricity itself.

That switch is called a transistor, and it is the single most manufactured object in human history, more transistors have been produced than grains of rice, corn, and wheat ever harvested, combined. But to understand the transistor, it helps to meet its simpler cousin first: the diode.

1.4 The Diode: A One-Way Street for Electricity

A diode is the simplest useful thing you can build out of doped silicon. It has exactly one job: let electric current flow through in one direction, and block it in the other.

Picture a one-way flap valve in a water pipe. Push water one way, the flap swings open and the water flows. Try to push the water back the other way, and the pressure slams the flap shut. Nothing gets through. A diode does the same thing to electric current.



Left: the water-pipe intuition. Right: the same idea in an actual circuit , flip the diode around and the bulb goes dark, because current can no longer complete the loop.

Why should you care about a one-way valve for electricity? Because it is the first example of a component making a decision. Current either flows or it doesn't, based purely on direction. Once you can make components decide things, you are one short step away from Boolean logic , the True/False world that every program you will ever write is quietly built on top of.



PYTHIA SAYS

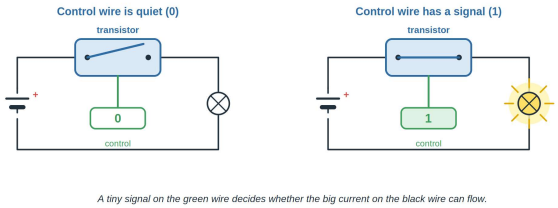
A diode's favourite exercise? One-way flow. It refuses to go both ways, and honestly, respect.

1.5 The Transistor: A Switch With No Moving Parts

A transistor takes the one-way idea of the diode and adds a third wire: a control wire. Think of it as a valve on a water pipe, except instead of turning a physical handle, you send a small electric signal down a side pipe, and that signal decides whether the main pipe is open or shut.

Concretely, most transistors have three connections:

- **Source** , where current can enter
- **Drain** , where current can leave
- **Gate** , the control wire. A voltage here decides whether current is allowed to flow between source and drain at all.



Same circuit, two states. No signal on the control wire, and the transistor behaves like a broken switch , nothing flows. A signal arrives, and it behaves like a closed switch , current flows and the bulb lights.

That is genuinely the whole idea. A transistor is a switch, but instead of your finger flipping it, another electrical signal does. And since the output of one transistor (current flowing or not) can become the control input of the next transistor, you can chain them together , signals controlling signals controlling signals , with no human finger involved anywhere in the chain.

Modern processors pack tens of billions of these switches onto a chip smaller than a fingernail, each one only a few nanometres across , for scale, a human hair is roughly 80,000 to 100,000 nanometres wide. Everything from here on is just this same idea, scaled up.



PYTHIA SAYS

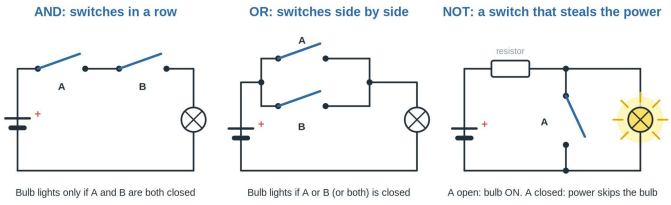
A modern high-end chip can have over 50 billion transistors on it. If each transistor were a grain of rice, you would need enough rice to bury a football field roughly waist-deep. Your phone does this while also sitting quietly in your pocket.

1.6 Switches Learn to Make Decisions: Logic Gates

A single switch can only do one thing , be on or off. But wire a few switches together in the right pattern, and the combination can answer a question. This is where electricity starts to look like logic.

AND, OR, and NOT with ordinary switches

Imagine a bulb, a battery, and some switches, all connected in a loop:

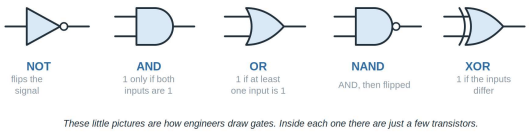


Three ways of wiring switches, and three different rules for when the bulb lights up.

- Put two switches in a row (in series), and the bulb only lights if both are closed. This is an AND , the current has only one path, and every switch on that path must be closed.
- Put two switches side by side (in parallel), and the bulb lights if either one is closed. This is an OR , the current has two possible paths, and it only needs one of them open.

- Wire a switch so that closing it diverts current away from the bulb instead of towards it, and you get a NOT , closing the switch turns the bulb off, opening it turns the bulb on. The output is the opposite of the input.

Engineers do not usually draw circuits like this to explain logic , that gets messy fast. Instead they use simple shorthand symbols, called logic gates, where the shape of the symbol tells you the rule:



The standard gate symbols. NAND is simply an AND with its answer flipped, shown by the small circle (called a "bubble") on the output.

Here is what each one does to two inputs, A and B, where 1 means "on/true" and 0 means "off/false":

Gate	Rule
AND	Output is 1 only when A = 1 and B = 1. Otherwise 0.
OR	Output is 1 when A = 1 or B = 1 (or both). Only 0 when both are 0.
NOT	Takes one input. Output is the opposite: 1 becomes 0, 0 becomes 1.
NAND	AND, then flipped. Output is 0 only when A = 1 and B = 1.
XOR	Output is 1 only when A and B are different from each other.

Here is a small but genuinely important fact: NAND gates alone are enough to build every other gate on this list, and therefore every digital circuit that has ever been built, including the one your phone is running right now. It is called a "universal gate" for exactly this reason. We will not prove that here either, but if you ever meet a hardware engineer at a party, mentioning it will buy you at least one more minute of their attention.



PYTHIA SAYS

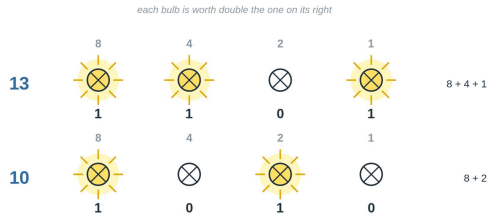
There are only two kinds of people in this world: those who understand binary, and those who don't.

(I'll wait.)

1.7 Bits: The Only Two Digits a Computer Needs

A transistor is either letting current through or not. A switch is either open or closed. A gate's output is either 1 or 0. Everything in a digital computer, at the very bottom, is a choice between exactly two states. We call each such choice a bit, short for "binary digit."

A single bit is not very expressive , it can represent on/off, true/false, yes/no, and nothing more. The trick is to line several bits up in a row and let each position stand for a different power of two, exactly the way each position in a normal decimal number stands for a different power of ten. In decimal, the number 231 means 2 hundreds, 3 tens, and 1 one. In binary, we just switch the powers from 10 to 2.



Four bulbs, worth 8, 4, 2, and 1. Light the ones whose value adds up to your number, and you have written that number in binary.

So 1101 in binary means 8 + 4 + 0 + 1 = 13, and 1010 means 8 + 0 + 2 + 0 = 10. This is not a different kind of number , it is the same familiar numbers, just written using a different set of digits. Everything your computer stores, whether it is a number, a letter of the alphabet, a photograph, or this very sentence, is eventually just a long string of these two digits, grouped and interpreted according to some agreed-upon convention.

- A group of 8 bits is called a byte. One byte can represent 256 different values (0 to 255) , which happens to be exactly enough to represent every character in the basic ASCII text standard, with room to spare.
- A kilobyte, megabyte, and gigabyte are (roughly) a thousand, a million, and a billion bytes , technically powers of 1024 rather than 1000, a small but famous source of confusion between hardware manufacturers and operating systems.



PYTHIA SAYS

There are 10 kinds of people in the world: those who understand binary, and those who don't. See what I did there? No? That's fair, it only works in binary.

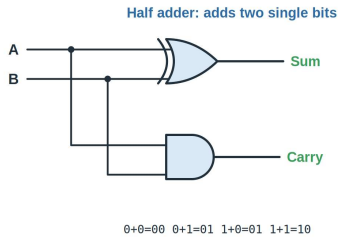
1.8 Gates Learn Arithmetic: Building an Adder

Logic gates answering true/false questions is a nice party trick, but computers are famous for arithmetic. The bridge between the two is shorter than you'd expect: addition, done in binary, is a logic problem in disguise.

Think about how you add two single binary digits by hand, the same way you were taught to add decimal digits in primary school, just with a much smaller multiplication table to memorise:

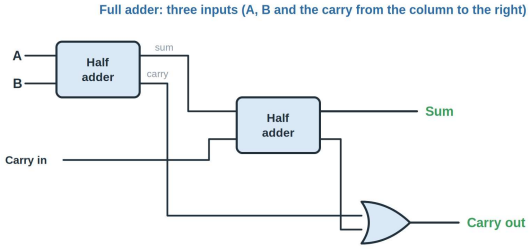
```
0 + 0 = 0     (sum 0, no carry)
0 + 1 = 1     (sum 1, no carry)
1 + 0 = 1     (sum 1, no carry)
1 + 1 = 10    (sum 0, carry 1  -- just like 5 + 5 = 10 in decimal)
```

Look closely at that table and you will notice two familiar patterns hiding inside it. The Sum column is 1 exactly when the two inputs differ , that's an XOR. The Carry column is 1 exactly when both inputs are 1 , that's an AND. Wire one XOR gate and one AND gate to the same two inputs, and you have built a circuit that adds two bits together. This little circuit even has a name: a half adder.



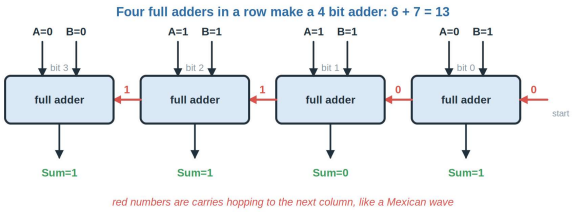
A half adder: one XOR gate for the sum, one AND gate for the carry. Two gates, and you can already add.

It's called a “half” adder because real addition, done column by column on paper, often needs to also add in a carry arriving from the column to its right. A half adder has nowhere to put that incoming carry. Chain two half adders together, with an OR gate to combine the two possible carries, and you get a full adder , a circuit that adds three bits at once: A, B, and a carry-in, producing a Sum and a carry-out.



A full adder, built from two half adders and one OR gate. This one circuit is the entire basis of binary addition.

Now for the satisfying part: line up four full adders side by side, and feed the carry-out of each one into the carry-in of its neighbour to the left, exactly like carrying a digit when you add on paper. The result is a circuit that adds two 4-bit numbers.



Four full adders chained together, adding 0110 (6) and 0111 (7) to get 1101 (13). The carries ripple leftward, column by column, which is why this design is called a ripple-carry adder.

Real processors add 32 or 64 bits at a time, not 4, and use cleverer designs than a simple ripple carry so they don't have to wait for the carry to travel all the way across the chip. But the underlying idea is exactly what you just saw. There is no separate “addition organ” hidden inside your CPU , addition is just a specific, fixed pattern of AND, OR, and XOR gates, wired once at the factory and never changed again.



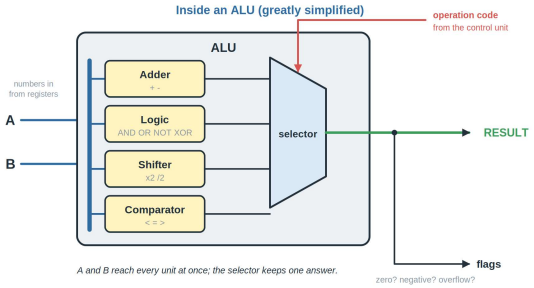
PYTHIA SAYS

Take a second here. A few paragraphs ago we had a rock. Now that rock can add. This is the entire miracle of computing in one chapter: nothing new was invented after the transistor. Everything else is just transistors, arranged patiently, in greater and greater numbers.

1.9 From Adder to ALU

Addition is one operation among many a processor needs: subtraction, comparison, shifting bits left or right (which, conveniently, is the same as multiplying or dividing by two), and the logic operations AND/OR/NOT/XOR themselves, so that your Python code's if statements have something to run on.

All of these live together in one region of the chip called the Arithmetic Logic Unit, or ALU. You can think of it as several small specialist circuits , an adder, a comparator, a shifter, a logic unit , all working on the same two input numbers simultaneously, with one final component, a selector (built, unsurprisingly, from more logic gates), choosing which one of those answers actually gets passed on, based on which operation was requested.



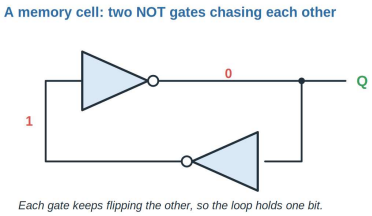
A simplified ALU. Every sub-unit computes its answer on every single clock tick, whether it's needed or not , the selector just decides which answer we keep.

Alongside the main result, an ALU also produces a handful of flags , single bits that record useful facts about the answer, such as whether it came out to exactly zero, whether it was negative, or whether the result was too large to fit and “overflowed.” These flags matter enormously: they are how a decision like if x > 0 in your Python code eventually becomes a real, physical yes/no signal inside the chip.

1.10 Remembering Things: How a Circuit Holds a Bit

Everything so far , gates, adders, the ALU , has been what engineers call combinational logic: the output depends only on the current inputs, right now, this instant. The moment the inputs change or the power is interrupted, the answer is gone. That is fine for calculating, but useless for remembering. A computer that could only calculate and never store anything would be a very fast, very forgetful calculator.

Memory needs a different trick: feedback. Take two NOT gates and wire the output of each one back around into the input of the other, forming a loop.



Two NOT gates chasing their own tails. Whatever value ends up on this loop, it keeps re-confirming itself, forever, as long as the power stays on.

Once you kick this loop into a particular state, it stays there. The first gate's output is 1, so it forces the second gate's output to 0, which in turn forces the first gate's output to stay at 1. Neither gate can independently escape the pattern the other is holding it in. This tiny stubborn loop is, essentially, one bit of memory. Real memory circuits (called latches and flip-flops) add a few more gates so you can deliberately overwrite the stored value when you want to, but the core idea is exactly this: a loop that argues with itself into staying put.

Wire millions or billions of these tiny memory loops into a grid, add circuitry to select one row and column at a time, and you have built RAM , Random Access Memory, the place where your running programs and their data live while the computer is switched on. “Random access” simply means any single memory location can be reached directly, in roughly the same amount of time, regardless of where it physically sits in the grid , unlike, say, a cassette tape, where you have to wind through everything before it to get to the part you want.

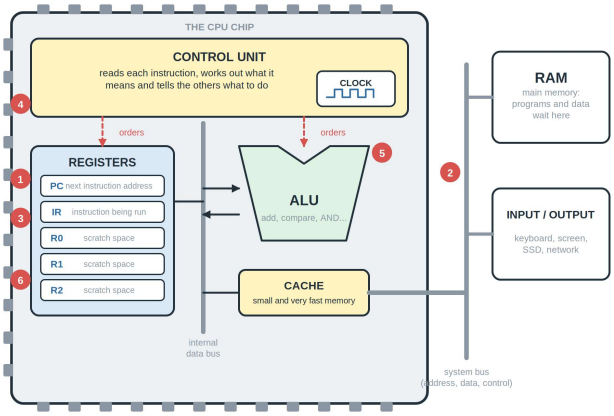


PYTHIA SAYS

RAM forgets everything the instant the power goes off, which is why your unsaved document vanishes in a power cut. Storage devices like SSDs and hard drives use a different kind of circuitry (SSDs still use transistors, just a special kind that can hold a charge without power) precisely so they don't have this problem. That trade-off , RAM is blazingly fast but forgetful, storage is slower but remembers , shapes almost every performance decision in computing.

1.11 Putting It All Together: Inside a CPU

We now have every ingredient: switches that make decisions (gates), circuits that calculate (the ALU), and circuits that remember (memory). A CPU (Central Processing Unit) is what you get when you package these together with one more essential ingredient , something to keep everyone working in the right order.

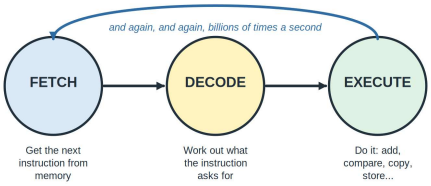


A heavily simplified map of a CPU chip. Real chips have far more circuitry , multiple cores, several layers of cache, branch predictors, and more , but every real CPU contains something recognisable as each of these six pieces.

Let's walk through the numbered pieces:

- **1. Registers** are the tiniest, fastest scraps of memory a CPU has , built from exactly the kind of feedback loop you just met, just very few of them, and placed as physically close to the ALU as engineering allows. A typical processor has only a few dozen of these, each holding one number at a time. Two of them have special jobs worth naming: the Program Counter (PC), which always holds the memory address of the next instruction to run, and the Instruction Register (IR), which holds the instruction currently being carried out.
- **2. The system bus** is the shared set of wires connecting the CPU to everything outside it , RAM, storage, keyboard, screen, network card. Think of it as the chip's road network: addresses, data, and control signals all travel along it.
- **3. The Instruction Register** holds the actual instruction the control unit is currently working on, freshly fetched from memory.
- **4. The control unit** is the CPU's supervisor. It does not do any arithmetic itself , its entire job is to read each instruction, work out what it means, and send the correct wake-up signals (shown here as dashed red “orders”) to the ALU and registers so they carry it out. It keeps time using the clock, a circuit that ticks at a fixed, extremely high frequency (a “3 GHz” processor ticks three billion times a second) so that every part of the chip stays in step, like a metronome for electrons.
- **5. The ALU** is the calculator we already built in section 1.9, now wired up to receive its two input numbers from the registers and send its answer back to them.
- **6. Cache** is a small pool of memory, built the same way as registers but with room for far more data, sitting between the CPU and the comparatively distant, comparatively slow RAM. Because programs tend to reuse the same data and instructions repeatedly in a short span of time, keeping a copy nearby in cache saves the CPU from making the much longer round trip to RAM every single time.

Everything the control unit does, over and over, follows the same three-step rhythm:



The fetch-decode-execute cycle. One trip around this loop, on a modern chip, can take well under a nanosecond.

- FETCH: the control unit looks at the Program Counter to find out where the next instruction lives in RAM, and copies that instruction into the Instruction Register.
- DECODE: the control unit figures out what the instruction is asking for , add these two registers, compare these values, jump somewhere else , and works out which parts of the chip need to be switched on to make it happen.
- EXECUTE: the relevant circuitry (usually the ALU, sometimes memory or an I/O device) actually does the work, and the Program Counter is updated to point at whatever instruction comes next.

Then it does it again. And again. Billions of times a second, for as long as the machine is switched on. There is no cleverness hiding anywhere in this loop , no part of a CPU "understands" what a Python for loop is, or what your variable names mean, or what your program is trying to achieve. It just fetches, decodes, and executes, one small, boring, completely mechanical instruction at a time, at a speed that makes the boredom irrelevant.



PYTHIA SAYS

And that's the whole staircase: sand becomes silicon, silicon becomes a diode, a diode becomes a transistor, transistors become gates, gates become adders and memory, and all of it together becomes a CPU that can fetch, decode, and execute. Every single thing you'll do in Python for the rest of this book , every variable, every loop, every function call , eventually turns into this exact cycle, running underneath you, whether you think about it or not.

You don't need to think about it while you code. But now, when someone says "computers just follow instructions," you'll know exactly how deep that sentence goes.

1.12 From Instructions to Programs

A single fetch-decode-execute step does something almost embarrassingly small: add two numbers, or move a number from one register to another, or check whether a value is zero. This tiny vocabulary is called a CPU's instruction set. Different families of processors (x86 in most laptops and desktops, ARM in most phones and increasingly in laptops too) have different instruction sets, which is one reason a program built for one cannot simply run on the other without translation.

A program, then, is nothing more than a very long, very precise shopping list of these tiny instructions, stored in memory in the order the control unit should carry them out. Writing software directly in this raw instruction form (called machine code, or one small step up, assembly language) is possible, and people did exactly that for

the earliest computers. It is also slow, tedious, and extremely easy to get wrong, because you are working at the level of individual registers and memory addresses rather than ideas.

This is exactly the gap that languages like Python exist to close. When you write a line like total = price * quantity, you are not writing machine instructions , you are writing something a human can read and reason about. A separate program (an interpreter, for Python) reads your code and, ultimately, triggers the right sequence of fetch-decode-execute cycles to make it happen. You get to think about the problem. The machine still only ever does one tiny thing at a time; it just does it staggeringly fast, and something else now handles the translation for you.

That translation, and the layer of thinking that has to happen before you even get to write total = price * quantity, is what the rest of this book is about.

Chapter 1 Questions

These are grouped by level: warm-ups to check you followed the chapter, exam-style questions you might see in a college paper, interview-style questions the kind an internship or placement interviewer might actually ask, and a few just-curious questions for if you want to go further. Answers to every question appear at the very end of the book, grouped by chapter , try answering first, honestly, before flipping there.

Warm-up

1. What is the difference between a diode and a transistor?
2. Why is a transistor described as “a switch with no moving parts”?
3. Name the three basic logic gates and describe what each one does.
4. What is a bit? What is a byte?
5. Write 1011 in binary as an ordinary decimal number.
6. What is the difference between a half adder and a full adder?
7. List the three steps of the fetch-decode-execute cycle in order.
8. What is the job of the control unit? Does it do arithmetic itself?

Typical exam-style questions

1. Explain, with a diagram, how AND and OR gates can be built using ordinary switches connected to a battery and a bulb.
2. Explain why NAND is called a “universal gate.” What does this mean in practice?
3. Draw and explain the working of a half adder. Extend your answer to describe how two half adders and an OR gate form a full adder.
4. Describe the fetch-decode-execute cycle with the help of a labelled diagram. What role does the clock play in this cycle?
5. Differentiate between RAM and cache memory in terms of speed, size, and position relative to the CPU.
6. What is meant by a “model of computation”? Briefly describe the Turing Machine and the RAM model.

Placement / interview style

1. Why can't a computer store negative numbers using a minus sign the way we write them on paper? (Hint: think about what a bit actually is, and look up “two's complement” if you get stuck.)
2. If NAND alone can build every other logic gate, why do real chips still use AND, OR, and NOT gates directly instead of only NAND gates?
3. A processor is advertised as running at 3.5 GHz. Roughly how long does one clock cycle take? Does a higher clock speed always mean a faster computer? Why or why not?
4. What is the difference between a CPU register and a variable in a Python program? Why can't Python variables just live in registers all the time?

5. Explain, in your own words, why cache memory improves performance even though it stores a copy of data that already exists somewhere else (in RAM).
6. What would happen, conceptually, if the feedback loop in a memory latch (section 1.10) lost power for even a fraction of a second?

Just curious

1. How small can a transistor actually get, and what happens when you try to make it smaller still?
2. If computers only understand 0s and 1s, how does a CPU know the difference between a number, a letter, and an instruction, when they might all be stored as the exact same pattern of bits?
3. Quantum computers are sometimes described as using “qubits” instead of bits. Based on what you now know about a bit, what do you think might be fundamentally different about a qubit? (You are not expected to know the answer yet , just to guess intelligently.)
4. Every general-purpose computer built on the model in this chapter is, in principle, equally powerful (recall the Church-Turing thesis). So why does a supercomputer exist at all, if a laptop can compute anything a supercomputer can?

CHAPTER 2

Problems, Algorithms, and the Problem-Solving Process

Chapter 1 was about the machine. This chapter is about you , specifically, about what has to happen inside your head before anything useful happens inside the machine. Most beginners open an editor and start typing the moment they read a problem. This chapter argues, gently but firmly, that this is almost always a mistake, and it will walk one single example all the way from a vague sentence in English to a working, tested, evaluated Python program, so you can see exactly what “thinking before typing” actually looks like.



PYTHIA SAYS

New chapter, same snake. This one has no diodes, but it has a running example I'm quite fond of , we're going to teach a computer to play a guessing game, and along the way you'll pick up every idea in this chapter's title. Stick with the one example. By the end you'll be able to trace it with your eyes closed.

2.1 Problems and Computational Problems

A problem, in the everyday sense, is simply a gap between where you are and where you want to be, with no immediately obvious way across. “I am hungry and there is no food in the house” is a problem. So is “I need to know which of these 500 email addresses have already unsubscribed.”

A computational problem is a problem with three properties that make it suitable for a machine to solve:

- **It has well-defined inputs** , the information you start with can be precisely stated (numbers, text, a list, an image, and so on).
- **It has a well-defined output** , you can precisely state what a correct answer looks like, even before you know how to get there.
- **There is a describable relationship between the two** , some rule, however complicated, connects a given input to its correct output.

“I am hungry” is not yet a computational problem , it's too vague. “Given the contents of my fridge (input), produce a list of meals I can cook without going to the shop (output), where a meal counts if every one of its ingredients appears in the fridge in sufficient quantity (rule)” is a computational problem. Most of the work in turning a real problem into a computational one is simply this: replacing vague nouns with precise ones.

2.2 What Exactly Is an Algorithm?

An algorithm is a finite sequence of well-defined, unambiguous steps that takes an input and produces the desired output. Every word in that sentence is doing work, so it's worth unpacking:

Property	What it means
Finite	It must end. “Keep trying numbers until you find the answer” is not an algorithm unless you can guarantee it eventually stops , otherwise it's just a plan to run forever.
Well-defined	Each step must be precise enough that there is no room for interpretation. “Add a bit of salt” is a recipe instruction; “add 5 grams of salt” is an algorithm instruction.
Unambiguous / definite	Given the same situation twice, the step means the same thing both times. No step should depend on a guess.
Effective	Every step must be something the machine (or person) carrying it out can actually do, using only the tools it already has.
Correct	For every valid input, it produces the right output. An algorithm that is fast but sometimes wrong isn't a working algorithm , it's a bug with good marketing.

Notice that nothing in this definition mentions Python, or even computers. A knitting pattern, a recipe, and the instructions for assembling flat-pack furniture are all algorithms in this sense (some, admittedly, fail the “unambiguous” test rather badly). Programming is simply the act of expressing an algorithm in a language a machine can carry out , the algorithm is the idea; the program is one particular way of writing that idea down.



PYTHIA SAYS

Why do programmers confuse Halloween and Christmas?

Because Oct 31 == Dec 25. Go on, convert them , I'll wait. (Octal 31 is 25 in decimal. Decimal 25 is also 25. You'll get there by Chapter 4.)

2.3 Problem-Solving Strategies, in Brief

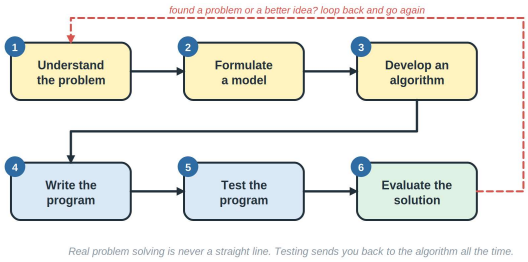
Once you understand a problem, there is usually more than one path to a working algorithm. You will meet each of these properly in later chapters, but it's worth knowing the family names now, the way it's useful to know “comedy” and “tragedy” exist before you've watched either.

Strategy	The idea
Brute force	Try every possibility until one works. Simple to design, often slow, but a perfectly good place to start.
Divide and conquer	Split the problem into smaller versions of itself, solve those, then combine the results. Our example this chapter uses exactly this idea.
Greedy	At each step, make whichever choice looks best right now, without reconsidering it later. Fast, but not always correct , sometimes the locally best choice is a globally bad one.

Strategy	The idea
Dynamic programming	Break a problem into overlapping smaller problems, and remember (“memoise”) answers you've already worked out, so you never solve the same small problem twice.
Backtracking	Build a solution piece by piece, and the moment you notice a piece cannot possibly lead anywhere useful, abandon it and try something else.

2.4 The Problem-Solving Process

Put the last three sections together and you get a process: understand what is being asked, describe it precisely (model it), design a method (algorithm), express that method to a machine (program), check that the expression actually works (test), and honestly judge how good the result is (evaluate). Six stages, and , this is the part beginners usually aren't told , the arrows between them are not one-way.



The six stages of problem solving. Notice the dashed line looping back to the start , finding a flaw at stage 5 sends you back to stage 1 or 3 far more often than any textbook diagram likes to admit.

We are now going to walk through all six stages, in order, using one running example: teaching a computer to play a number-guessing game. It's small enough to fully understand, but it contains a genuinely important algorithmic idea, and we'll use it to introduce three different ways of describing an algorithm along the way: plain English, a flowchart, and pseudocode.

2.5 Stage 1: Understanding the Problem

Here is the problem, stated the way a friend might actually say it to you:

> *“I'm thinking of a number between 1 and 100. You guess it, and after each guess I'll tell you if you need to go higher or lower. Try to find it in as few guesses as possible.”*

This sounds simple, and it is , but notice how much is still left unsaid. Before designing anything, a careful problem-solver would ask:

- Are both endpoints, 1 and 100, allowed answers, or is the range strictly between them?
- Are we guaranteed the friend is telling the truth, and that a number actually was chosen in advance? (Surprisingly, real puzzle-setters sometimes cheat , we will come back to this in the Testing stage.)
- What exactly do we want as output: just the number, or also how many guesses it took?

- Is there a maximum number of guesses allowed, or do we simply want to minimise them?

For this chapter, let's settle these deliberately, the way you would when writing a specification for yourself:

```
INPUT:   a hidden whole number, secret, with 1 <= secret <= 100
         (we cannot see secret directly , only feedback on guesses)
OUTPUT:  the value of secret, and how many guesses it took
FEEDBACK, after each guess g, is exactly one of:
         "correct"   if g == secret
         "higher"    if g <  secret   (secret is higher than our guess)
         "lower"     if g >  secret   (secret is lower than our guess)
```

Try a manual example before designing anything, just to build intuition. Suppose the secret is 37. If you guess 50, you're told “lower.” Guess 25, you're told “higher.” Guess 37, and you're told “correct.” Three guesses. Could you have done it in two? Could you always guarantee some maximum number of guesses, no matter what the secret turns out to be? Hold onto that question , we'll answer it properly in the Evaluation stage.



PYTHIA SAYS

This precondition/postcondition style , stating exactly what you're given and exactly what you must produce , might feel like overkill for a guessing game. It is, a little. But the habit is the whole point: the day you do this for a genuinely hard problem, half the fight is already won, because you can no longer quietly misunderstand your own question.

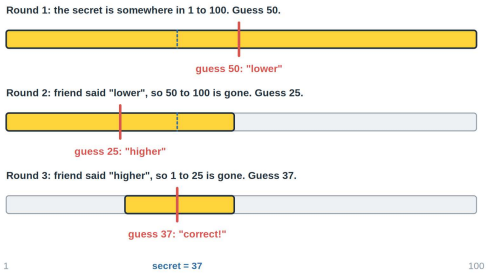
2.6 Stage 2: Formulating a Model

Modelling means deciding how to represent the problem so that reasoning about it becomes easier, deliberately throwing away detail that doesn't matter. The friend, the spoken feedback, the fact that this is a “game” , none of that is essential. What's essential is much narrower: at any moment, we know the secret number lies somewhere inside a range of possible values, and each guess either finds it or shrinks that range.

So our model needs exactly three pieces of information, kept up to date as we go:

- **low** , the smallest value the secret could still possibly be
- **high** , the largest value the secret could still possibly be
- **guess** , our current attempt, some value between low and high

Initially, we know nothing beyond what the problem told us, so low = 1 and high = 100 , the secret could be anywhere in that whole range. Every time we get feedback, we use it to shrink the range: “higher” means the secret must be above our guess, so we can raise low; “lower” means it must be below, so we can lower high. This is our whole model: a shrinking window of possibility, closing in on one number.



The model in action, secret = 37. Each guess cuts the remaining possibilities roughly in half. Three guesses, and the window has closed to a single number.



PYTHIA SAYS

This shrinking-window idea has a name you'll meet constantly from here on: binary search. It works on guessing games, sure, but it's also how you'd find a word in a paper dictionary (open roughly to the middle, decide which half to search next, repeat), and it's one of the first "real" algorithms most people learn. You're not just solving a guessing game right now , you're pre-loading a very famous idea.

2.7 Stage 3a: The Algorithm, in Plain English First

Before reaching for a diagram or any code, it's worth forcing yourself to write the algorithm out as a sequence of plain English instructions. This step gets skipped constantly, and skipping it is exactly how people end up staring at a flowchart or a page of code with no idea what it's supposed to be doing. Here is our guessing strategy, written the way you might explain it to a younger sibling:

```
1. Start by assuming the secret could be anywhere from 1 to 100.
2. Guess the number exactly in the middle of your remaining range.
3. Ask whether that guess is correct, too high, or too low.
4. If it's correct, stop , you're done.
5. If told to go higher, forget every number at or below your guess;
   the secret must live in the upper part of your remaining range.
6. If told to go lower, forget every number at or above your guess;
   the secret must live in the lower part of your remaining range.
7. Go back to step 2, using the new, smaller range.
```

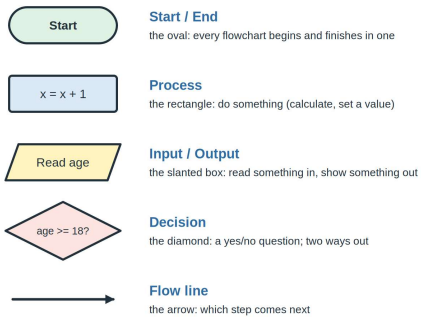
Read that back and check it against our earlier manual example (secret = 37): guess 50, told lower, range becomes 1–49; guess 25, told higher, range becomes 26–49; guess 37, correct, stop. It matches. That agreement between a hand-worked example and your plain-English steps is a small, cheap, and extremely valuable sanity check , always do it before moving on.

2.8 Stage 3b: Drawing It as a Flowchart

Plain English is readable, but it hides its own structure , step 7 sends you "back to step 2," and in a longer algorithm, tracking which step loops back to which other step, in your head, gets error-prone fast. A flowchart is

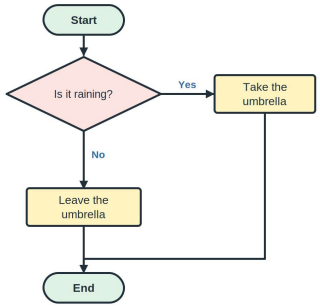
a diagram that makes this structure visible: each kind of action gets its own shape, and arrows show exactly which step leads to which.

Here are the shapes we'll use, and what each one means:



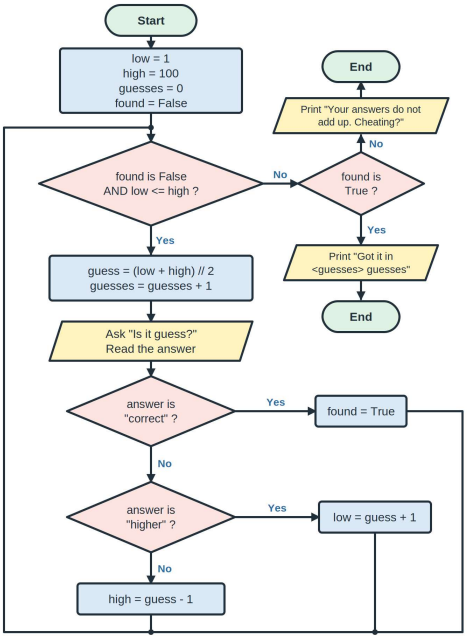
The handful of shapes you need for almost any flowchart. Everything else is just arrows connecting them.

The most important shape, by far, is the diamond, because it's the only one with two exits. Every other shape has exactly one arrow leading out of it , the diamond is where an algorithm actually makes a decision. Here is a tiny warm-up flowchart before we tackle the real one, for a much smaller everyday problem: deciding whether to take an umbrella.



A flowchart barely bigger than the plain-English version , but notice how unmistakable the two paths are, compared to describing them in a sentence.

Now the real one. Translating our seven plain-English steps into a flowchart, being careful to route both the “correct” exit and the loop-back arrow properly:



The full guessing algorithm as a flowchart. Follow the arrows with your finger for the secret = 37 example and check it against section 2.6 , every loop, every decision, should match exactly.

A few things to notice, because they're easy to miss on a first read and important on every flowchart you'll ever draw:

- There is exactly one Start and, here, two possible Ends , flowcharts are allowed multiple exits, as long as every path eventually reaches one.
- The large loop on the left (the diamond asking found is False AND low <= high?) is what turns our seven “go back to step 2” instructions into an actual repeating structure , this loop is the entire engine of the algorithm.
- There's an extra path on the right that plain English never mentioned: what if low ever becomes greater than high? That can only happen if the feedback we were given was inconsistent , in other words, if someone lied to us, or made a mistake. A careful flowchart makes even impossible-sounding situations explicit instead of quietly ignoring them.



PYTHIA SAYS

That last bullet point is the diamond that catches a cheating friend. If your low ever overtakes your high, mathematically no number can satisfy all the hints you were given , someone changed their mind about the secret partway through, on purpose or not. Flowcharts are wonderfully petty that way: draw one properly and it will catch your friends lying to you. Software has never sounded more useful.

2.9 Stage 3c: Writing It as Pseudocode

A flowchart is great for seeing structure at a glance, but it gets unwieldy to draw for anything much bigger, and it doesn't look like any real programming language, which makes the final jump to code feel larger than it needs to

be. Pseudocode solves both problems: it's structured like real code (with clear steps, loops, and conditions) but written in plain, informal language, free of any particular language's exact punctuation rules. Nobody's pseudocode looks identical to anyone else's, and that's fine , its only job is to be unambiguous to a human reader.

Here is our flowchart, rewritten as pseudocode:

```
ALGORITHM GuessMyNumber
  low <- 1
  high <- 100
  guesses <- 0
  found <- False

  WHILE found is False AND low <= high DO
      guess <- (low + high) divided by 2, rounded down
      guesses <- guesses + 1
      ask "Is it <guess>?" and read the answer

      IF answer is "correct" THEN
          found <- True
      ELSE IF answer is "higher" THEN
          low <- guess + 1
      ELSE   (answer was "lower")
          high <- guess - 1
      END IF
  END WHILE

  IF found is True THEN
      OUTPUT secret was <guess>, found in <guesses> guesses
  ELSE
      OUTPUT the answers given were inconsistent
  END IF
```

Compare this side by side with the flowchart. The large decision diamond became the WHILE line. The three-way branch (correct / higher / lower) became an IF / ELSE IF / ELSE block. Nothing new was invented , pseudocode is simply a more compact, more linear way of writing exactly the same algorithm the flowchart already described. This is very much intentional: by the time you reach pseudocode, the hard thinking is already finished. What's left is closer to translation than to problem-solving.



PYTHIA SAYS

Notice the arrow "<-" for assignment in the pseudocode above, instead of an equals sign. That's deliberate. In mathematics, x = x + 1 is a contradiction , no number equals itself plus one. In programming, x = x + 1 (or x <- x + 1) means something completely different: take the current value of x, add one, and store that back into x. Pseudocode's arrow is a small habit that keeps this distinction honest in your head before you meet Python's = and get used to overloading the symbol.

2.10 Stage 4: Writing the Program

With a tested-by-hand plain English description, a flowchart, and pseudocode all agreeing with each other, writing the actual Python is close to mechanical. This is exactly the point of the earlier stages , by the time you reach this one, you are translating a decision you have already fully made, not making it up as you type. Here is the pseudocode, translated line for line:

```
import random
 def guess_my_number(secret, low=1, high=100):
    """
    Play the higher/lower guessing game against `secret`.
    Returns the number of guesses it took, or None if the
    feedback we were given turns out to be inconsistent.
    """
    guesses = 0
    found = False
     while not found and low <= high:
        guess = (low + high) // 2      # rounds down
        guesses = guesses + 1

        if guess == secret:
            found = True
        elif guess < secret:
            low = guess + 1             # go higher next time
        else:
            high = guess - 1            # go lower next time
     if found:
        return guesses
    else:
        return None                # the feedback did not add up
 if __name__ == "__main__":
    secret_number = random.randint(1, 100)
    result = guess_my_number(secret_number)
    print(f"Secret was {secret_number}, found in {result} guesses.")
```

A few small, deliberate choices are worth pointing out. The function takes secret as a parameter instead of asking the real user “Is it correct?” at a keyboard , this makes the function usable both by an actual human player and, in the next section, by an automated test that already knows the right answer and can check our logic without a human sitting there typing responses all day. The // operator performs integer division, rounding down, exactly matching “divided by 2, rounded down” from the pseudocode. And not found reads almost exactly like the English “found is False” from our flowchart's diamond.



PYTHIA SAYS

Why did the Python programmer name every variable secret? Because they didn't want anyone to know what was going on. I'll see myself out.

2.11 Stage 5: Testing the Program

Testing is not “run it once and see if it looks right.” It's a deliberate search for the ways your program could be wrong, using cases chosen specifically because they're likely to expose a flaw. A good rule of thumb: design your test cases before you feel confident, not after , confidence is exactly the feeling that makes you stop looking.

For our guessing function, a solid set of tests should include the smallest possible secret, the largest possible secret, a secret right in the middle where the very first guess is correct, and , since we deliberately built in a check for it , a case of contradictory feedback.

```
def test_guess_my_number():
    # every possible secret should be found, and always within
    # 7 guesses, since 2**7 = 128 comfortably covers 1..100
    for secret in range(1, 101):
        result = guess_my_number(secret)
        assert result is not None, f"failed to find {secret}"
        assert result <= 7, f"{secret} took too many: {result}"

    # boundary cases, tested individually for clarity
    assert guess_my_number(1)   is not None
    assert guess_my_number(100) is not None
    assert guess_my_number(50)  == 1     # dead centre, first guess

    print("All tests passed!")

test_guess_my_number()
```

Look closely at the very first test: instead of picking a handful of secrets by hand, it checks every single one from 1 to 100. This is called exhaustive testing, and it's only practical here because the input space is small , for a problem with billions of possible inputs, you'd have to fall back on carefully chosen representative cases instead. When exhaustive testing is possible, it's the strongest evidence you can get, short of a mathematical proof, that your algorithm is correct.

Notice also that we asserted result <= 7, not just “result is not None.” That number, 7, is not arbitrary , we predicted it in section 2.6 by reasoning about how fast the range shrinks, and now the test is holding the code to that prediction. If a future change to the code (say, an off-by-one slip in low = guess + 1) ever made some secret take 8 guesses instead, this test would catch it immediately, long before a human noticed anything felt slower than usual.

As for the deliberately inconsistent-feedback case: our function can't manufacture bad feedback on its own here, since it's playing against a known secret, so proper testing of that branch would call for a small stand-in , something pretending to be a lying player , which is a technique (sometimes called a mock) you'll meet properly in a later chapter on testing. For now, it's enough to know the branch exists, deliberately, and that it is the kind of thing a thorough test suite would eventually need to reach.



PYTHIA SAYS

A programmer's partner says, "While you're at the shop, get a loaf of bread. And if they have eggs, get a dozen." The programmer comes back with 12 loaves of bread. When asked why, they said: "They had eggs." Testing is what stops that programmer from shipping the 12 loaves to production.

2.12 Stage 6: Evaluating the Solution

Passing every test tells you the program is correct. It tells you nothing about whether it's good. Evaluation asks a different, harder set of questions, mostly about cost and clarity rather than correctness.

Start with efficiency. Every guess our algorithm makes eliminates roughly half of the remaining possibilities , that's the whole point of aiming at the middle of the range. Starting from 100 possible numbers, one guess leaves about 50, the next leaves about 25, then about 12, 6, 3, 1. Six or seven guesses, and we're always down to a single possibility, no matter which of the 100 numbers was secretly chosen. Compare that to the obvious alternative: guessing 1, then 2, then 3, and so on upward. On average that brute-force approach needs 50 guesses, and in the worst case (the secret is 100) it needs all 100. Our halving strategy needs at most 7, guaranteed, for any secret in the range , an enormous improvement, and it's a direct, practical reward for the divide-and-conquer thinking we did back in the modelling stage.

This kind of comparison is usually written using Big-O notation, which describes how the number of steps grows as the input size (call it n , here, the width of the range) grows, ignoring constant details that don't matter for large n. The straightforward guess-upward approach is O(n): double the range, and you double the worst case. Our halving approach is $O(\log_2 n)$: double the range, and you add only one extra guess. That gap , between an algorithm that scales linearly and one that scales logarithmically , is one of the most important and recurring ideas in this entire subject, and you've now met it through a game rather than a formula.

Efficiency isn't the only lens, though. A full evaluation also asks:

- **Clarity** , could another programmer read this function and understand it in under a minute? Ours is short and its variable names (low, high, guess) map directly onto the model from section 2.6, which is exactly why we chose them there and not later.
- **Robustness** , what happens on unexpected input? Our function would currently behave oddly if given a secret outside 1–100, since it would never appear inside the search range at all, and the loop would simply run out without ever setting found to True. A more defensive version might check this explicitly up front and fail with a clear message rather than a silent wrong answer.
- **Generality** , our function already accepts low and high as parameters with defaults, rather than hard-coding 1 and 100 inside the function body, so it quietly works for a guessing game from 1 to 1000 with no changes needed. Small design choices like this, made early, tend to pay for themselves later.

A good evaluation ends by asking whether you should go back and change anything. Here, the honest answer is: for this exact problem, probably not , the algorithm is already close to the best possible, and the code is short and clear. But that conclusion is only trustworthy because you did the work to check it, rather than assumed it.

That habit, more than any single technique in this chapter, is what separates someone who writes code that happens to work from someone who actually knows why it works.



PYTHIA SAYS

Six stages, one small guessing game, and you've quietly met binary search, Big-O notation, exhaustive testing, and defensive programming, all without a single import beyond random. That's really the whole trick of this book: pick problems small enough to fully understand, and let them carry ideas far bigger than themselves.

Chapter 2 Questions

As before: warm-ups, exam-style, placement/interview-style, and a few curious ones for the road. Answers for every question in this book are collected at the very end.

Warm-up

1. What three properties turn an everyday problem into a computational problem?
2. List the five properties an algorithm must have.
3. Name the six stages of the problem-solving process, in order.
4. What is the difference between a plain-English algorithm, a flowchart, and pseudocode? Why bother with more than one?
5. In a flowchart, which shape is the only one with two exits, and why does that matter?
6. In our pseudocode, what does the symbol “<-” mean, and why is it not simply written as “=”?
7. What is the purpose of a test case like guess_my_number(50) == 1 in our test suite?

Typical exam-style questions

1. Define an algorithm and explain each of its defining properties with a short example of your own.
2. Explain, with reference to the guessing game example, the difference between “understanding the problem” and “formulating a model.” Why are they treated as separate stages?
3. Draw a flowchart for an algorithm that finds the largest of three given numbers. Use the standard flowchart symbols correctly.
4. Write pseudocode for an algorithm that checks whether a given whole number is prime.
5. Explain what Big-O notation represents. Using our guessing game as an example, explain the difference between O(n) and O(log n) behaviour.
6. What is exhaustive testing? Under what circumstances is it a practical testing strategy, and when does it stop being one?

Placement / interview style

1. Our guessing algorithm always finds the secret in at most 7 guesses. Where exactly does the number 7 come from, and how would that number change if the range were 1 to 1,000 instead of 1 to 100?
2. Suppose a colleague's version of guess_my_number used high = guess (instead of guess - 1) whenever the feedback was “lower.” What would go wrong, and under what specific circumstance would you actually notice the bug at runtime?
3. Why does the code check not found and low <= high, using AND, rather than just low <= high on its own? What would break if that first condition were removed?
4. You are asked to test a sorting function. It passes on every test case you tried. Describe three categories of input you would deliberately test next, and explain what each one is trying to catch.

5. Explain the difference between an algorithm being “correct” and an algorithm being “good.” Give an example of an algorithm that is correct but not good.
6. A friend says, “I don't need to design an algorithm on paper first, I can just start writing Python and fix it as I go.” Under what conditions might that actually be a reasonable approach, and under what conditions is it likely to backfire?

Just curious

1. Our algorithm always guesses exactly the midpoint of the remaining range. Can you think of a situation, in a real (not purely mathematical) guessing game, where guessing something other than the exact midpoint might actually be the smarter move?
2. The number 7 keeps showing up for our 1–100 guessing game because $2^7 = 128 \geq 100$. Can you find the general rule connecting the size of the range to the guaranteed maximum number of guesses?
3. We modelled the problem using just three variables: low, high, and guess. Can you think of a real-world problem where choosing the right model , deciding what to keep and what to throw away , is actually the hardest part of solving it, harder than the algorithm itself?
4. Every algorithm in this chapter eventually became Python. Can you think of an algorithm you already know (perhaps from outside computing entirely) that would resist being written as a step-by-step procedure at all?